

Edge AI-Enabled Autonomous Firefighting and Hazard Detection Robot for Smart Emergency Response

Ashik Mahmud

Dept. of CSE

Ahsanullah University of Science and Technology

Dhaka, Bangladesh

20220104021

Farhan Labib

Dept. of CSE

Ahsanullah University of Science and Technology

Dhaka, Bangladesh

20220104029

Maisha Ahmed

Dept. of CSE

Ahsanullah University of Science and Technology

Dhaka, Bangladesh

20220104050

Prottoy Roy Deep

Dept. of CSE

Ahsanullah University of Science and Technology

Dhaka, Bangladesh

20220104099

Abstract—Most low-cost firefighting robots detect fire with threshold-based flame, gas, or infrared sensors and coordinate sensing, navigation, and suppression through simple conditional logic. This paper presents FireBot, a vision-only autonomous ground robot that performs the complete detect–track–approach–suppress cycle on a Raspberry Pi 4 without any cloud dependency. A YOLOv8n detector fine-tuned on a two-class (fire, smoke) dataset reaches an mAP@0.5 of 0.894 on a held-out test split (0.909 for fire) and, exported to the NCNN framework for CPU-only inference, processes frames from a Pi Camera mounted on a pan–tilt gimbal. A finite-state controller searches an eight-position scan grid, centres the flame with incremental pan/tilt corrections, steers a four-wheel differential-drive chassis using short PWM pulses, and uses the width of the detected bounding box as a monocular proximity cue in place of an ultrasonic range sensor. When the flame is close, centred for three consecutive frames, and aligned with the chassis heading, a relay-driven pump sprays water through a servo-swept hose. We describe the hardware, circuit, and control design, report tuning results including unsuccessful configurations, and report 30 successful end-to-end trials from 1–3 m and five approach angles ($\pm 60^\circ$, $\pm 30^\circ$, 0°), with an approach time of about 1.5–3 min. We also discuss the limitations of bounding-box-based proximity estimation and low-pressure suppression.

Keywords—Edge AI, fire detection, YOLO, NCNN, Raspberry Pi, autonomous robot, visual servoing, Internet of Things

I. Introduction

Indoor fires spread quickly, and early suppression at the point of ignition can prevent a small hazard from becoming a structural fire. Mobile firefighting robots aim to reach that point faster and more safely than a human responder. Existing low-cost designs, however, typically rely on flame or infrared threshold sensors, which give a binary fire/no-fire output with no information about the fire’s position in the scene or the certainty of the detection [1], [3], [8]. Deep-learning detectors

such as YOLO have been shown to run in real time on embedded hardware [9], [11], but these systems are usually deployed as stationary or aerial detect-and-alert units rather than on robots that can physically respond [10].

This project closes that gap with FireBot, a ground robot that uses a single camera as its only exteroceptive sensor. An earlier prototype also carried an HC-SR04 ultrasonic sensor on a scanning servo for range measurement; that subsystem was removed in the final design because its noisy readings slowed the control loop and a faulty sensor could fail without any indication. Proximity is now inferred from the apparent size of the detected flame, which also lowers cost and part count. The contributions of this work are:

- an on-device YOLO/NCNN fire-detection pipeline on a Raspberry Pi 4 that outputs confidence-scored bounding boxes used directly for control;
- a closed-loop state machine (SEARCH, TRACK, APPROACH, HOLD, SPRAY, REACQUIRE) that couples pan–tilt visual tracking with pulse-based chassis steering;
- a vision-only spray trigger that combines bounding-box width, image centring, and chassis alignment, together with a sweeping hose to widen suppression coverage;
- an experimental account of the tuning process, including configurations that did not work and the reasons for them.

The rest of the paper is organized as follows. Section II reviews related work. Section III describes the system architecture and control method. Section IV covers the prototype and experimental setup. Sections V and VI present and discuss the results, and Section VII concludes.

II. Related Work

II-A. IoT-Based Firefighting Robots

IoT firefighting robots share a common architecture: a microcontroller or single-board computer, a sensor suite (flame, gas, temperature, or infrared), a mobile base, a suppression mechanism (water pump or CO₂), and a wireless link for monitoring. Perumal *et al.* added a night-vision camera to address visibility loss in smoke or darkness [1]. Anantha Raj and Srivani placed the robot inside the Internet of Robotic Things paradigm, treating it as one node of a networked fire-safety system [2]. Ambafi *et al.* combined sensors, motor control, a water pump, and a web camera interface with manual override [3], and a related Raspberry Pi system added obstacle avoidance with automatic/manual modes [4]. Kanwar and Agilandeewari selected water or CO₂ suppression according to the detected fire type [8]. Other works used Raspberry Pi control with IoT-based monitoring [5], [6], and a Raspberry Pi Pico W prototype used a Grid-EYE thermal sensor with CO₂ suppression [7].

II-B. Edge-AI Fire Detection

A separate line of work moves from sensor thresholds to learned visual detection on embedded hardware. Yuliawati *et al.* deployed YOLOv8 on a Jetson platform with an IP camera, reporting 96% accuracy and 97.9% mAP [9]. Titu *et al.* ran lightweight detectors on drone-mounted edge hardware [10], and Lu *et al.* proposed FCMI-YOLO, reducing parameters by 40% while improving small-flame detection [11]. Outside fire response, Santos *et al.* combined YOLOv7, a Jetson Xavier NX, a Raspberry Pi 4, a camera, and LIDAR for autonomous inspection [12].

II-C. Research Gap

Table I summarizes the comparison. Sensor-based robots [1]–[4], [8] lack graded, position-aware detection; edge-AI detectors [9]–[11] lack a mobile suppression platform; and robotic edge-AI navigation [12] has not been applied to fire suppression. No reviewed system integrates learned detection, visual tracking, approach, and suppression in one on-device loop.

III. Proposed System and Methodology

III-A. System Overview

Fig. 1 shows the architecture. All perception and control run on a Raspberry Pi 4; no external server or cloud API is used. The camera is the only sensor that observes the environment, and the controller acts on the chassis motors, the camera gimbal, the pump relay, and the hose servo.

III-B. Hardware Components

- **Compute:** Raspberry Pi 4 Model B in a fan-cooled case.
- **Vision:** Raspberry Pi Camera Module v1.3 on a two-servo pan-tilt bracket (pan GPIO 16, tilt GPIO 10).
- **Mobility:** four-wheel differential drive with one DC motor per wheel, driven by two L298N dual H-bridges (one

per side). Unlike the checkpoint prototype, the enable pins are now driven with 1 kHz PWM for per-motor speed control.

- **Suppression:** water pump in an on-board reservoir, switched by a relay module (GPIO 9, active HIGH); the hose outlet is mounted on a front servo (GPIO 13) that sweeps it horizontally.
- **Power:** a 3S LiPo (11.1 V) feeds both L298N modules; a buck converter provides a regulated 5 V rail for the Pi, servos, and relay. All grounds are common.

Figs. 2–4 show the assembled robot.

III-C. Circuit Design

Fig. 5 and Table II give the pin allocation. Motors M1 (front-left) and M2 (rear-left) share one L298N; M3 (front-right) and M4 (rear-right) share the other. Servo pulses are generated through the pigpio daemon to avoid the jitter of software PWM.

III-D. Fire Detection Model and Inference Pipeline

The detector is YOLOv8n (about 3.0 M parameters) initialised from COCO-pretrained weights and fine-tuned on the Home Fire Dataset [13], a publicly available YOLO-format dataset for early indoor fire detection with two classes (*fire*, *smoke*); the YOLO26n base checkpoint was also considered. Training used an input size of 640 × 640, 50 epochs, batch size 8, the Ultralytics automatic optimizer choice with a linearly decaying learning rate, and default augmentation (HSV jitter, translation, scaling, horizontal flip, and mosaic, which was disabled for the last 10 epochs). Training took about 60 min on a GPU. The best weights were exported to NCNN (FP32, 12.1 MB) for CPU-only inference on the Pi.

Fig. 6 shows the per-frame pipeline. Each 640 × 480 camera frame is rotated by 180° to compensate for the camera mounting, resized to 640 × 640, converted to RGB, and normalized. The network outputs 8,400 candidate boxes with a score for each class. The controller uses the *fire* score only: candidates below a confidence of 0.40 are discarded, overlapping boxes are removed by a custom non-maximum suppression (IoU 0.45), and the highest-confidence box is taken as the target.

III-E. Autonomous Control

The controller is the state machine in Fig. 7.

SEARCH: The gimbal visits eight pan positions in two tilt rows, dwelling 5 s at each while running detection. Tilt commands are mapped from a logical range [−0.20, 0.70] to the servo range [−1, 1] by

$$s_{\text{tilt}} = -1 + 2 \frac{t - t_{\min}}{t_{\max} - t_{\min}}, \quad (1)$$

after clamping t to $[t_{\min}, t_{\max}]$, so the calibrated mechanical limits are never exceeded.

TRACK: With frame centre (c_x, c_y) and box centre (f_x, f_y) , the errors are $e_x = f_x - c_x$ and $e_y = f_y - c_y$. If $|e_x| > 40$ px, pan moves by a fixed step of 0.02 toward the flame;

TABLE I
COMPARISON OF EXISTING STUDIES WITH THE PROPOSED SYSTEM.

Work / focus	Difference from FireBot	Limitation / gap	What FireBot adds
Night-vision robot [1]	Night/thermal viewing instead of learned detection	No AI localization, centring, or approach logic	YOLO detection with confidence and position/size
IoRT robot [2]	Intelligence in the network rather than on the robot	Connectivity alone gives no visual understanding	Fully on-device inference and decisions
IoT smart robot [3]	Closest system, but detection is sensor-based	Limited visual localization and tracking	Pan-tilt tracking, aligned approach, autonomous spray
Monitoring/control robot [4]	Focus on remote monitoring and control	No continuous fire-centring mechanism	Automatic acquisition and centring
Detection/control robot [6]	Relies on remote/manual control	Limited autonomous navigation to the fire	Autonomous steering from visual error
Pico W mini robot [7]	Thermal sensing, no camera-based AI	No visual position/size information	Visual localization with hose sweep
Cloud IoT robot [8]	Remote-operation oriented	Cloud link does not solve targeting	Closed-loop, cloud-free control
Edge-AI detectors [9]–[11]	Detect-and-alert only	No mobile suppression platform	Detection embedded in an acting robot

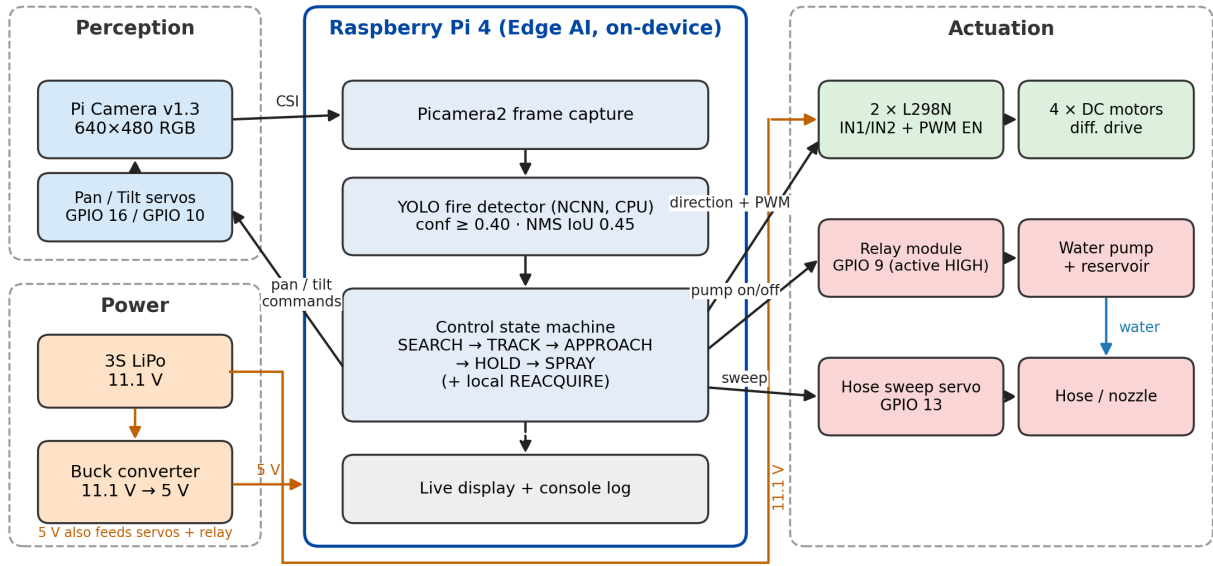


Fig. 1. FireBot system architecture (vision-only, fully on-device).

TABLE II
GPIO ALLOCATION (BCM NUMBERING).

GPIO	Pin	Function
4	7	M3 enable (PWM), front-right
5 / 6	29 / 31	M4 IN1 / IN2, rear-right
9	21	Pump relay (active HIGH)
10	19	Camera tilt servo
13	33	Hose sweep servo
14	8	M1 enable (PWM), front-left
16	36	Camera pan servo
17 / 27	11 / 13	M1 IN1 / IN2, front-left
18	12	M2 enable (PWM), rear-left
19	35	M4 enable (PWM), rear-right
22 / 23	15 / 16	M2 IN1 / IN2, rear-left
24 / 25	18 / 22	M3 IN1 / IN2, front-right

otherwise, if $|e_y| > 120$ px, tilt moves by 0.02. Horizontal error is corrected first because it drives steering. The larger

vertical tolerance allows for the flame appearing low in the image at close range.

APPROACH: Because the camera, not the chassis, points at the fire, the pan angle p encodes the heading error. If $|p - p_0| > 0.25$ (with $p_0 = 0$ straight ahead), the robot turns in place with a short pulse at 100% duty (0.065 s left, 0.06 s right); otherwise it drives forward for 0.18 s with per-motor duties of 55/55/90/65% (M1–M4) to compensate for motor mismatch. The chassis stops for 0.10 s before every pulse.

Proximity and HOLD: With no range sensor, the fire is considered close when the bounding-box width $w \geq 450$ px (about 70% of the frame width). The chassis then performs a final alignment correction if $|p - p_0| > 0.25$, and otherwise holds position until the flame is inside the centring window for three consecutive frames.

SPRAY: The pump runs for 20 control epochs with the chassis stopped while the hose servo sweeps in steps of 0.20

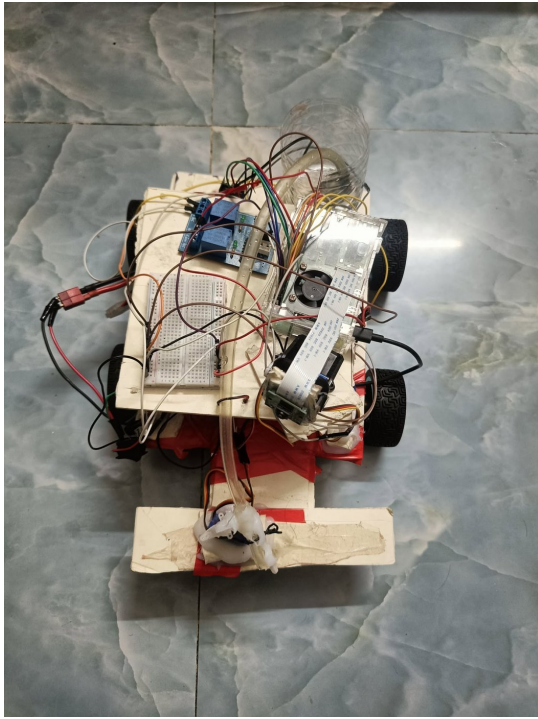


Fig. 2. Top view: Raspberry Pi 4, pan-tilt camera, relay module, breadboard, rear reservoir, and front hose servo.

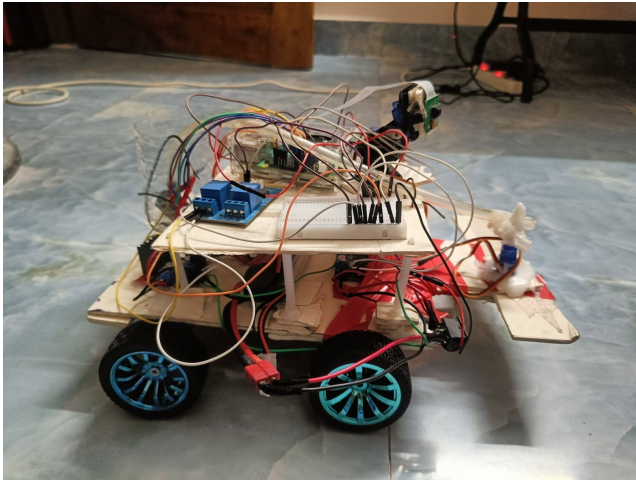


Fig. 3. Side view of the two-deck chassis with the camera gimbal and front hose-sweep servo.

between -1.0 and $+0.4$; the camera continues to track. The pump stops when the epochs are complete or the fire is lost.

REACQUIRE: After 15 consecutive frames without detection, the gimbal checks four points around the last position (± 0.15 pan, ± 0.10 tilt). If none contains fire, the robot returns to SEARCH.

III-F. Safety and Privacy

Frames are processed in memory and never transmitted, so normal operation raises no privacy concerns. Any runtime exception in the main loop stops all motors and the pump

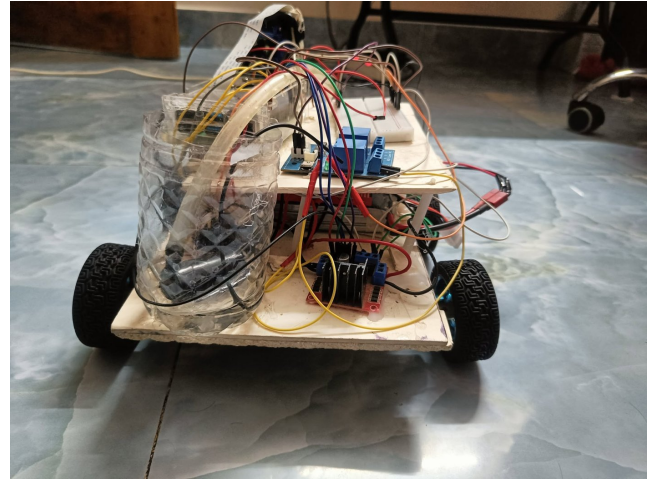


Fig. 4. Rear view: water reservoir with pump, and L298N motor driver on the lower deck.

before the loop recovers, and a shutdown routine stops PWM, releases the servos, and cleans up the GPIO so that no actuator is left energized.

IV. Prototype Implementation and Experimental Setup

IV-A. Prototype Development

Before chassis integration, the Pi, power board, and gimbal were validated on an open bench rig (Fig. 9). A keyboard-driven calibration tool jogged each servo in 1° and 10° steps and recorded its mechanical centre and usable range, which define the pan limits $[-0.50, 1.00]$ and the tilt limits above. The final robot uses a two-deck foam-board chassis: motor drivers and battery on the lower deck, and the Pi, relay, and breadboard on the upper deck (Figs. 2–4).

IV-B. Test Environment and Dataset

For safety, tests were run indoors on a tiled floor with fire images (not open flames) as targets. The detector was trained and evaluated on the Home Fire Dataset [13] using its training, validation, and test splits; the test split contains 897 labelled fire instances and 689 smoke instances. During bench testing on the robot, every detection was logged with its timestamp, confidence, box coordinates, and servo angles, giving 2,092 on-robot fire detections over two days of testing. Fig. 10 shows a typical detection.

V. Results

V-A. Detection Model

Fig. 8 shows training converging smoothly over 50 epochs with no sign of over-fitting: validation losses track the training losses, and the metrics plateau after about epoch 35. The drop in classification loss at epoch 41 coincides with mosaic augmentation being switched off. Table IV summarizes detection performance on the validation split (final epoch) and on the held-out test split.

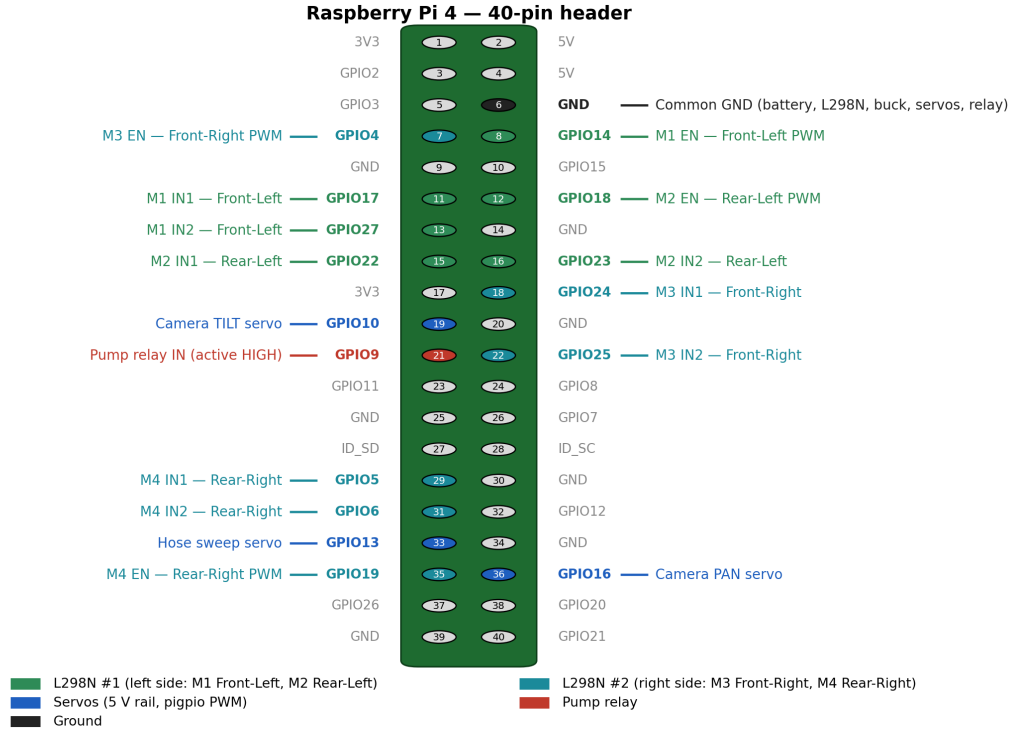


Fig. 5. Raspberry Pi 4 GPIO wiring diagram of the final (vision-only) FireBot.

TABLE III
UNSUCCESSFUL OR REVISED CONFIGURATIONS AND THE RESULTING CHANGES.

Configuration tried	Observed problem	Change / outcome
HC-SR04 ultrasonic ranging on a scanning servo	Noisy distance readings had to be filtered and re-checked, and the extra servo scan slowed the control loop. A broken or disconnected sensor also failed silently: the robot had no way to tell a faulty reading from a real one.	Sensor and servo removed; bounding-box width used as a monocular proximity cue. The robot is now vision-only, with fewer parts, lower cost, and no silent sensor failure mode.
Close-range threshold 400 px Forward pulse 0.15 s Left turn pulse 0.07 s Turning at reduced PWM duty	Robot began spraying too far from the flame Approach too slow; progress per cycle small Left turns overshot the target Wheels stalled from static friction during in-place turns	Raised to 450 px. Increased to 0.18 s. Reduced to 0.065 s (right: 0.05 s $\rightarrow$ 0.06 s). Turns run at 100% duty; turn size controlled by pulse time.
Equal duty on all motors Camera auto-exposure / white balance defaults Pump pressure Platform reliability	Robot drifted during forward motion Intermittent detection dropouts when exposure changed near the flame Spray works but has weak reach SD-card failures, LiPo cell imbalance, and boot failures	Per-motor duties 55/55/90/65%. Diagnosed and patched in the capture settings. Unresolved; hose sweep added to widen coverage. Boot moved to a USB card reader; battery rebalanced.

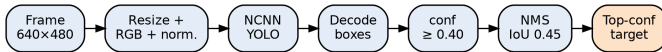


Fig. 6. Per-frame detection pipeline.

On the test split, the F1 score over both classes peaks at 0.85 at a confidence of 0.412 (Fig. 11); this is why the deployment threshold was set to 0.40. At that threshold, fire precision is about 0.92 and recall about 0.82. The normalized confusion matrix (Fig. 12) shows that 91% of labelled fire instances are detected as fire, 6% are confused with smoke, and 3% are missed; for smoke, 87% are correct, 9% are labelled as fire,

and 4% are missed. The background column contains many low-confidence false positives because Ultralytics builds the matrix from the validator’s very low confidence cut-off; most

TABLE IV
DETECTION PERFORMANCE OF THE FINE-TUNED YOLOV8N.

Split / class	Precision	Recall	mAP@0.5
Validation, all	0.919	0.892	0.939
Test, all [†]	0.89	0.82	0.894
Test, fire [†]	0.92	0.82	0.909
Test, smoke [†]	0.87	0.82	0.879

[†]P and R read from the test curves at confidence 0.41.

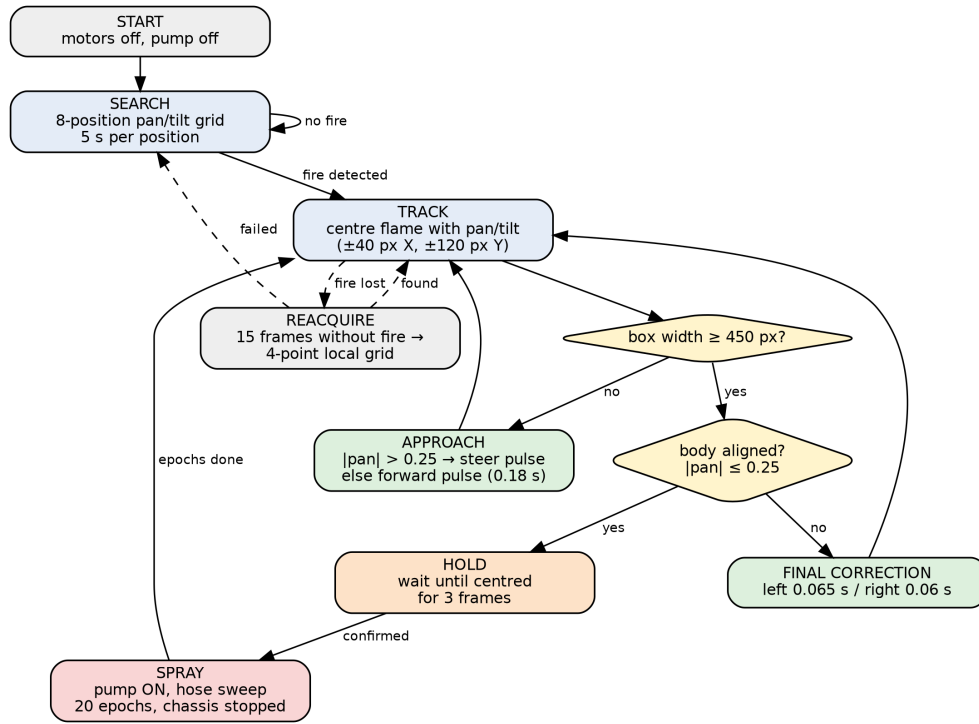


Fig. 7. FireBot control state machine.

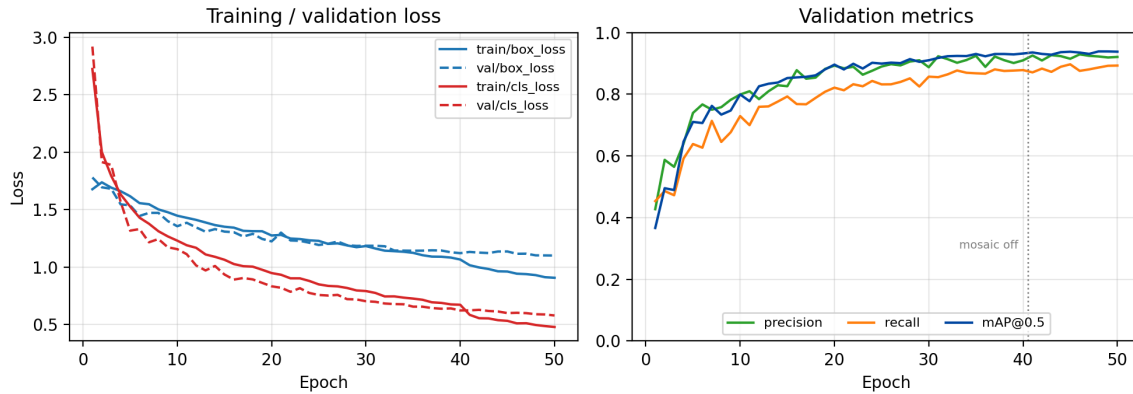


Fig. 8. YOLOv8n training: losses (left) and validation metrics (right) over 50 epochs.

of these are removed by the 0.40 threshold used on the robot. Fig. 13 shows example predictions on test images.

V-B. On-Robot Detection

Fig. 14 shows the confidence of the 2,092 fire detections logged on the robot. The mean confidence is 0.76 (median 0.80); 94% of detections exceed 0.5 and 73% exceed 0.7, so the flame is usually detected well above the 0.40 threshold. The median bounding-box width in these logs is 186 px, compared with the 450 px close-range trigger. On-device frame rate was not benchmarked separately; the control loop is dominated by servo settling and motion pulses rather than inference (Table VI).

V-C. Final Configuration

Table V lists the tuned parameters of the final system.

V-D. Tuning History and Unsuccessful Configurations

Table III summarizes configurations that were tried and changed, and why.

V-E. System-Level Behaviour

In indoor tests with fire-image targets, the robot detected the fire during the search sweep, centred it with the gimbal, approached with alternating turn and forward pulses, stopped at close range, and sprayed. Thirty end-to-end trials were run: ten from each starting distance (1, 2, and 3 m), with the flame placed at five angles relative to the robot's initial heading

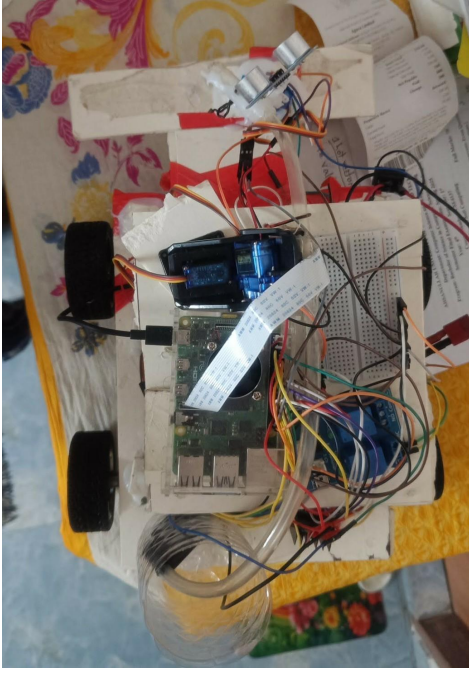


Fig. 9. Bench-level integration of the Pi 4, gimbal, buck converter, and LiPo pack.

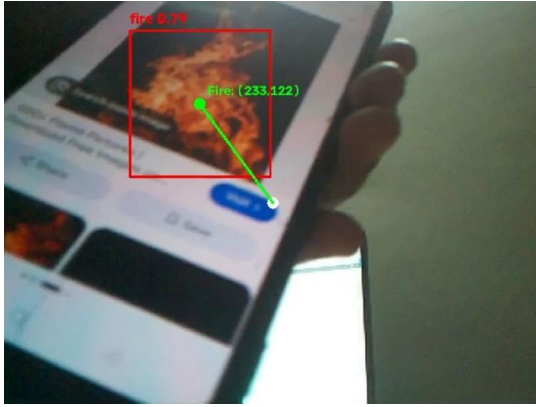


Fig. 10. Example detection with bounding box and estimated centre.

TABLE V
TUNED PARAMETERS OF THE FINAL SYSTEM.

Parameter	Value
Confidence / NMS IoU threshold	0.40 / 0.45
Centring tolerance (x, y)	40 px, 120 px
Pan / tilt step	0.02 / 0.02
Close-range trigger (box width)	≥ 450 px of 640
Spray confirmation	3 centred frames
Heading tolerance $ p - p_0 $	0.25
Forward pulse	0.18 s
Turn pulse (left / right)	0.065 s / 0.06 s at 100%
Target-loss limit	15 frames
Spray duration	20 epochs
Hose sweep	step 0.20, range $[-1.0, 0.4]$

(-60° , -30° , 0° , $+30^\circ$, and $+60^\circ$; two trials per angle at each distance). A trial was counted as successful when the

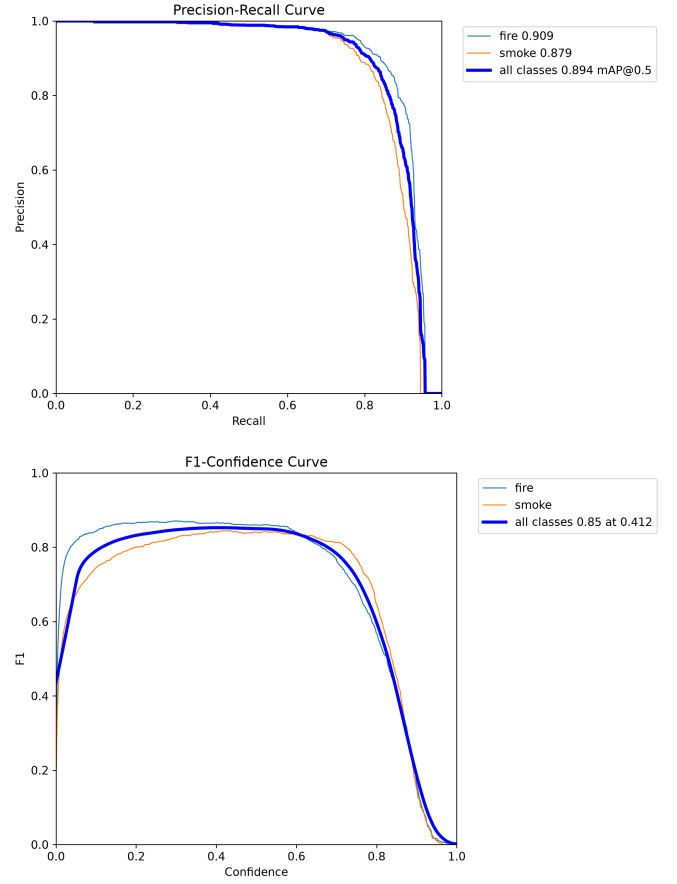


Fig. 11. Test-split precision-recall curve (top) and F1-confidence curve (bottom).

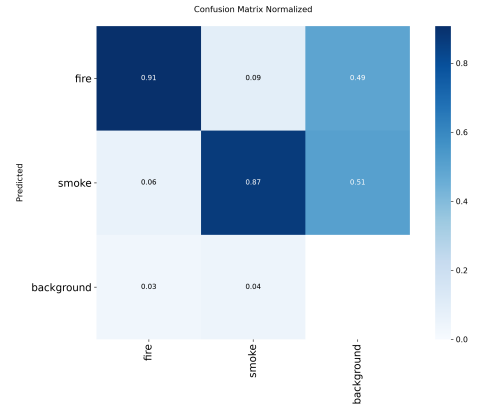


Fig. 12. Normalized confusion matrix on the test split.

robot completed the full search-track-approach-hold-spray sequence and sprayed water onto the target. Because the targets were images, these trials validate detection, navigation, alignment, and spray triggering, but not extinguishing performance. All 30 trials were successful (Table VI).

The approach is slow because every movement is a short



Fig. 13. Model predictions with confidence on test images.

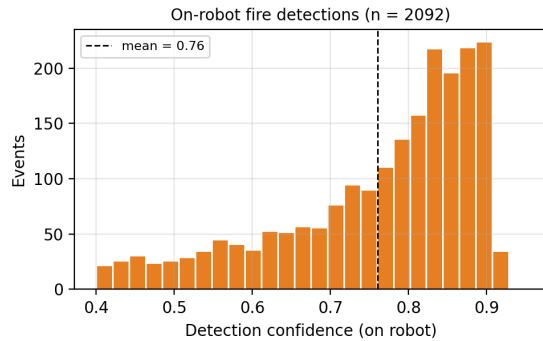


Fig. 14. Confidence of fire detections logged on the robot.

TABLE VI
END-TO-END TRIALS: SUCCESS RATE AND APPROACH TIME VERSUS
STARTING DISTANCE.

Start dist.	Successful	Time to reach fire	Avg. speed
1 m	10/10	≈1.5 min (90 s)	≈1.1 cm/s
2 m	10/10	≈2.6 min (156 s)	≈1.3 cm/s
3 m	10/10	≈3.0 min (180 s)	≈1.7 cm/s
Total	30/30		

stop-and-go pulse: each cycle includes a 0.10 s stop, a 0.18 s drive or turn pulse, a 0.30 s servo settle for each gimbal correction, and one inference pass. The robot therefore re-measures the fire position after every small move. The average speed rises with distance (1.1, 1.3, and 1.7 cm/s for 1, 2, and 3 m), which shows that part of the time is spent on the initial search and the final alignment near the fire rather than on forward travel. A least-squares line through the three points gives about 52 s of roughly fixed time (initial search and final alignment) plus about 45 s per metre of travel, i.e. an effective driving speed of about 2.2 cm/s once the robot is moving toward the fire.

VI. Discussion

Vision-only proximity. Using bounding-box width removes the ultrasonic sensor’s noise and its silent failure mode, avoids the time spent scanning it with a servo, and reduces the

hardware to one low-cost sensor. Its weakness is that box width depends on flame size as well as distance: a large fire triggers the spray from farther away and a small candle-sized flame requires a closer approach. The 450 px threshold was tuned for the test targets and would need recalibration, or a size-independent cue, for other fire sizes.

Pulse-based steering. Short pulses separated by stops make each motion small enough that the camera can re-measure the error before the next move. The cost is a slow approach (about 1.5–3 min for 1–3 m, Table VI), which is too slow for a real emergency; continuous proportional drive with an adaptive pulse length is the main planned improvement, and the asymmetric left/right pulse times show that the drivetrain is not symmetric.

Detector limitations. Test mAP@0.5 (0.894) is lower than validation mAP@0.5 (0.939), which is the expected gap on unseen images but also shows the model is not perfect. The model also outputs a smoke class, but the controller currently ignores it, so smoke-only events do not trigger any response. Fire and smoke are confused in 6–9% of instances, and bright reflections or orange objects remain possible false positives. The three-frame confirmation before spraying reduces but does not eliminate this risk. Finally, stretching the 640×480 frame to 640×640 without letterboxing distorts the aspect ratio compared with training.

Suppression. The small pump is the main limitation. The hose sweep increases coverage but not reach, so suppression is only expected to be effective for small flames at close range. Extinguishing a real flame was not tested in the reported trials, which used fire images for safety; controlled real-flame trials are the main item of future work.

Scan grid. Two scan positions request values outside the calibrated limits (pan -0.60 and tilt 1.00); these are clamped to -0.50 and 0.70 , so the upper scan row effectively runs at tilt 0.70 .

VII. Conclusion and Future Work

FireBot shows that the full detect–track–approach–spray cycle can run on a Raspberry Pi 4 using only a camera, an NCNN-optimized YOLO detector, and a simple state machine, with no cloud dependency. Future work includes a higher-pressure pump, fusing a thermal array or time-of-flight sensor for size-independent range, using the existing smoke output for early warning, letterboxed pre-processing, proportional (rather than fixed-step) gimbal control, and an IoT alert channel that notifies occupants without moving inference off the robot.

Project Resources

Demonstration video: <https://youtu.be/AtvzNyzKIjw>
Source code: <https://github.com/NoobCrow/FireBot>

Individual Contributions

Table VII lists each member’s work and the background research done for that task; Fig. 15 shows the overall contribution split.

TABLE VII
INDIVIDUAL CONTRIBUTIONS AND TASK-SPECIFIC BACKGROUND
RESEARCH.

Member	Contribution	Background research
Farhan Labib 20220104029	Overall AI pipeline, detection, tracking and control state machine, camera/servo and hose integration, testing; managed hardware components	Edge deployment of YOLO fire detectors and model optimization [9], [11]; NCNN CPU inference on Raspberry Pi
Ashik Mahmud 20220104021	AI/model assistance and testing; managed hardware components	Lightweight fire detection on edge hardware [10]; YOLOv8n training and evaluation
Prottoy Roy Deep 20220104099	Camera/servo hardware assistance; prepared the report	Pan-tilt tracking and pump-based suppression in IoT robots [3], [8]
Maisha Ahmed 20220104050	Testing, documentation and support; literature review	Survey of IoT firefighting robots and edge-AI robotics [1], [2], [4]–[7], [12]

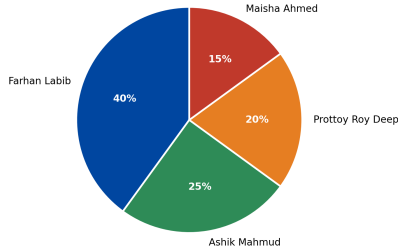


Fig. 15. Overall percentage contribution of team members.

References

- [1] K. A. P. Perumal, M. A. M. Ali, and Z. H. Yahya, "Fire fighter robot with night vision camera," in *Proc. IEEE 15th Int. Colloq. Signal Process. Its Appl. (CSPA)*, 2019, pp. 270–274, doi: 10.1109/CSPA.2019.8696077.
- [2] P. Anantha Raj and M. Srivani, "Internet of robotic things based autonomous fire fighting mobile robot," in *Proc. IEEE Int. Conf. Comput. Intell. Comput. Res. (ICCIC)*, 2018, pp. 1–4.
- [3] J. G. Ambafi *et al.*, "Development of a novel IoT-based smart firefighting robot for real-time fire detection and suppression," *Current J. Appl. Sci. Technol.*, vol. 44, no. 5, pp. 88–97, 2025.
- [4] K. Rambabu, S. Siriki, D. Chupernechitha, and C. Pooja, "Monitoring and controlling of fire fighting robot using IoT," *Int. J. Eng. Technol. Sci. Res.*, 2018.
- [5] R. Sarath Kumar, J. Hariharan, R. S. Revanth, K. R. Prasanth, and J. Lokesh, "Automatic fire fighting robot using RPI," in *Proc. 2nd Int. Conf. Smart Electron. Commun. (ICOSEC)*, 2021.
- [6] M. Sawant, R. Pagar, D. Zutshi, and S. Sadhukhan, "Fire detection and controlling robot using IoT," in *Advances in Machine Learning and Computational Intelligence (Proc. ICMLCI 2019)*, S. Patnaik, X.-S. Yang, and I. K. Sethi, Eds. Singapore: Springer, 2021, pp. 567–577, doi: 10.1007/978-981-15-5243-4_52.
- [7] X. Dang, "Design and prototype of a mini firefighting robot using Raspberry Pi Pico W," B.Eng. thesis, 2025.
- [8] M. Kanwar and L. Agilandeewari, "IoT based fire fighting robot," in *Proc. 7th Int. Conf. Reliability, Infocom Technol. Optim. (ICRITO)*, 2018.
- [9] D. Yulawati, S. Saleh, and T. M. Zaini, "AIoT untuk deteksi dini kebakaran menggunakan integrasi komputasi tepi dan kamera IP" [AIoT for early fire detection using edge computing and IP camera integration], *Teknika*, vol. 18, no. 2, pp. 455–465, 2024.
- [10] M. F. S. Titu, M. A. Pavel, M. K. O. Goh, H. Babar, U. Aman, and R. Khan, "Real-time fire detection: Integrating lightweight deep learning models on drones with edge computing," *Drones*, vol. 8, no. 9, p. 483, 2024, doi: 10.3390/drones8090483.
- [11] J. Lu, Y. Zheng, L. Guan, B. Lin, W. Shi, J. Zhang, and Y. Wu, "FCMI-YOLO: An efficient deep learning-based algorithm for real-time fire detection on edge devices," *PLOS ONE*, vol. 20, no. 8, e0329555, 2025, doi: 10.1371/journal.pone.0329555.
- [12] R. Santos, M. Silva, R. Lucas Santos, E. Klippel, and R. A. R. Oliveira, "Towards autonomous mobile inspection robots using edge AI," in *Proc. 18th Int. Conf. Softw. Technol. (ICSOT)*, 2023, pp. 555–562, doi: 10.5220/0011972200003467.
- [13] PengBo00, "Home fire dataset: A YOLO-ready dataset for early detection of indoor fires," Kaggle, 2025. [Online]. Available: <https://www.kaggle.com/datasets/pengbo00/home-fire-dataset>